

DESAFIO TÉCNICO

Vaga: Analista de Desenvolvimento

Prazo de submissão: até 15/08/2026, às 12h

1. Contexto

Este desafio faz parte do processo seletivo para a vaga de Analista de Desenvolvimento na Fadex. O objetivo é avaliar, em um cenário prático, a capacidade do(a) candidato(a) de projetar, implementar e documentar uma solução de software funcional, com um nível de inovação e iniciativa acima do CRUD tradicional.

O desafio foi desenhado para ser resolvido com as tecnologias que o(a) candidato(a) já domina (backend em Java/Spring Boot, Node.js/TypeScript ou Python/Django, entre outras), sem favorecer uma stack específica.

2. Projeto proposto: Central de Chamados com Triagem Inteligente

Desenvolva uma API REST para uma central de chamados internos (helpdesk) que utilize um componente de inteligência artificial para triagem automática das solicitações, além de indicadores em tempo real ambos elementos centrais da avaliação, e não apenas diferenciais.

A ideia é simular um cenário real de inovação interna: em vez de o solicitante escolher manualmente categoria e prioridade, o próprio sistema sugere essa classificação a partir do texto da solicitação, e a equipe de suporte é notificada instantaneamente quando surge algo urgente.

2.1 Entidades mínimas

- Usuário: nome, e-mail, senha (com hash), papel (ADMIN ou SOLICITANTE).
- Chamado: título, descrição, categoria, prioridade (BAIXA/MÉDIA/ALTA), status (ABERTO, EM_ANDAMENTO, RESOLVIDO, FECHADO), solicitante, responsável (opcional), origem da classificação (IA ou manual), datas de criação e atualização.
- Comentário/Histórico: registro de interações e mudanças de status em cada chamado (autor, texto, data).

2.2 Funcionalidades obrigatórias

1. Autenticação: cadastro e login de usuários com emissão de token (JWT ou equivalente).
2. Autorização: apenas ADMIN pode listar todos os chamados e reatribuir responsáveis; SOLICITANTE só visualiza e gerencia os próprios chamados.
3. CRUD completo de chamados (criar, listar com filtros por status/prioridade/categoria, visualizar detalhe, atualizar status, excluir/cancelar).
4. Triagem automática por IA: ao criar um chamado, o sistema deve sugerir categoria e prioridade a partir do texto da descrição (uso livre de API de IA gratuita, modelo local leve, ou heurística própria bem justificada o critério é a solução funcionar e estar bem explicada, não a sofisticação do modelo). O ADMIN pode aceitar ou corrigir a sugestão.
5. Indicadores em tempo real: um endpoint (e idealmente uma tela) que atualiza automaticamente via WebSocket, Server-Sent Events ou polling curto mostrando contagem de chamados por status/prioridade e alertando quando um chamado de prioridade ALTA é aberto.
6. Adição de comentários/histórico a um chamado existente.
7. Validações de regras de negócio (ex.: não permitir reabrir chamado fechado; campos obrigatórios; e-mail único).

3. Requisitos técnicos

3.1 Obrigatórios

- Código publicado em repositório público no GitHub, com histórico de commits (não aceitar commit único "projeto final").
- README.md com: descrição do projeto, tecnologias utilizadas, instruções claras de instalação e execução local (passo a passo), e como popular/testar a API (coleção Postman/Insomnia ou exemplos de requisições com curl).
- Persistência em banco de dados relacional (PostgreSQL, MySQL ou SQLite aceitos).

- Tratamento de erros com respostas HTTP e mensagens adequadas (400, 401, 403, 404, 500).
- Organização do código em camadas (ex.: controller/service/repository ou equivalente da stack escolhida).

3.2 Diferenciais (não obrigatórios, mas valorizados)

- Interface web simples (React, Angular ou Next.js) consumindo a API e exibindo o painel em tempo real.
- Detecção de chamados duplicados/similares (ex.: comparação por similaridade de texto/embeddings).
- Containerização com Docker / docker-compose para subir a aplicação com um único comando.
- Testes automatizados (unitários e/ou de integração).
- Documentação da API com Swagger/OpenAPI.
- Deploy funcional (ex.: Render, Railway, Vercel) além do repositório.

3.3 Diretrizes de acesso a IA e segurança

- Caso não tenha acesso a nenhuma API de IA gratuita (muitas exigem cadastro com cartão), é aceitável simular a triagem com uma função determinística/mock que demonstre a arquitetura da integração (ex.: onde a chamada real entraria). O foco da avaliação é o desenho da solução, não o modelo em si.
- Nenhuma chave de API, senha ou segredo deve ser commitado no repositório. Utilize variáveis de ambiente e inclua um arquivo .env.example com as chaves esperadas (sem valores reais).
- O README deve conter ao menos um usuário de teste (ex.: e-mail e senha de um ADMIN e de um SOLICITANTE) já cadastrado via seed/migration, para agilizar a avaliação.

4. Entregáveis

1. Link do repositório público no GitHub.
2. README completo conforme item 3.1.
3. (Opcional) Link de ambiente publicado, se houver deploy.

4. Envio do link por e-mail ou pelo canal indicado até o prazo final (15/08/2026, 12h), informando o horário de conclusão.

5. Critérios de avaliação

Critério	Peso	O que será observado
Funcionalidade	25%	Requisitos obrigatórios implementados e funcionando de fato.
Qualidade do código	20%	Organização, legibilidade, nomenclatura, separação de responsabilidades.
Triagem por IA e tempo real	20%	Solução de classificação automática implementada e funcionando; qualidade do mecanismo de atualização em tempo real.
Modelagem de dados	15%	Coerência do modelo relacional e das regras de negócio implementadas.
Boas práticas de Git	10%	Histórico de commits significativo, uso de branches (se houver).
Documentação	5%	Clareza do README e facilidade para reproduzir o projeto localmente.
Diferenciais	5%	Itens do item 3.2 implementados além do mínimo exigido.

6. Observações finais

- O componente de IA não precisa ser sofisticado: uma integração simples com uma API gratuita (ex.: Hugging Face Inference API, Google Gemini free tier, OpenAI) ou até uma heurística baseada em palavras-chave são aceitáveis, desde que a escolha esteja justificada no README.
- Não é esperado um sistema completo ou pronto para produção o foco é demonstrar raciocínio de arquitetura, capacidade de aprender/integrar uma tecnologia nova sob prazo e organização do trabalho.
- É permitido o uso de bibliotecas, frameworks e ferramentas de IA para apoio ao desenvolvimento o que será avaliado é o resultado final e a compreensão do candidato sobre o que foi construído.
- Em caso de dúvidas sobre o escopo, entre em contato com a equipe de recrutamento pelo canal informado no e-mail de convite.

Checklist de avaliação pós-submissão (uso interno)

Checklist a ser preenchido pela equipe de recrutamento durante a análise de cada repositório entregue, servindo de registro padronizado e comparável entre candidatos.

Categoria	Item avaliado	Conforme?
Autenticação e autorização	Cadastro de usuário funcional com hash de senha	☐ Sim ☐ Não
	Login retorna token válido (JWT ou equivalente)	☐ Sim ☐ Não
	Rotas protegidas exigem token válido	☐ Sim ☐ Não
	Permissões por papel (ADMIN x SOLICITANTE) aplicadas corretamente	☐ Sim ☐ Não
CRUD de chamados	Criação de chamado com validação de campos obrigatórios	☐ Sim ☐ Não
	Listagem com filtros por status/prioridade/categoria	☐ Sim ☐ Não
	Visualização de detalhe, atualização de status e exclusão/cancelamento	☐ Sim ☐ Não
	Regra de não reabertura de chamado fechado	☐ Sim ☐ Não
	E-mail de usuário validado como único	☐ Sim ☐ Não
Comentários / histórico	Endpoint de adição de comentário/interação	☐ Sim ☐ Não
	Histórico exibido em ordem cronológica	☐ Sim ☐ Não
Triagem por IA	Mecanismo de sugestão automática de categoria e prioridade implementado	☐ Sim ☐ Não
	Abordagem (API, modelo local, heurística ou mock) justificada no README	☐ Sim ☐ Não
	ADMIN pode aceitar/corrigir a sugestão da IA	☐ Sim ☐ Não
	Nenhuma chave de API/secredo commitado no repositório	☐ Sim ☐ Não
Tempo real	Endpoint de indicadores por status/prioridade disponível	☐ Sim ☐ Não
	Atualização automática implementada (WebSocket, SSE ou polling curto)	☐ Sim ☐ Não



	Alerta específico para chamados de prioridade ALTA	☐ Sim ☐ Não
Qualidade de código	Organização em camadas (controller/service/repository ou equivalente)	☐ Sim ☐ Não
	Nomenclatura consistente e legível	☐ Sim ☐ Não
	Tratamento de erros com status HTTP adequados (400/401/403/404/500)	☐ Sim ☐ Não
Modelagem de dados	Modelo relacional coerente com as entidades propostas	☐ Sim ☐ Não
	Relacionamentos, chaves e constraints corretamente definidos	☐ Sim ☐ Não
Git e versionamento	Histórico de commits granular (não é um commit único)	☐ Sim ☐ Não
	Mensagens de commit compreensíveis	☐ Sim ☐ Não
Documentação	README com descrição, tecnologias e passo a passo de instalação	☐ Sim ☐ Não
	Credenciais/seed de teste disponibilizadas	☐ Sim ☐ Não
	Coleção Postman/Insomnia ou exemplos de requisições (curl)	☐ Sim ☐ Não
Diferenciais	Frontend consumindo a API / Docker / testes / Swagger / deploy / detecção de duplicados	☐ Sim ☐ Não
Entrega	Repositório público e acessível	☐ Sim ☐ Não
	Submissão recebida até 15/08/2026, 12h	☐ Sim ☐ Não